🤖 AI Virtual Engineering Team

Powered by Claude (claude-sonnet-4-6) · Each persona builds on the previous one's work

Feature Request

Build a pipeline service in Python that accepts a GitHub repository URL and a feature description, clones the repo, analyzes the existing codebase structure, and generates a ready-to-merge pull request with implementation code, tests

🚀 Run the Team

🏗️ Architect

Architecture: AI-Powered PR Generation Pipeline

Component Breakdown

1. REST API Layer (`FastAPI`)

- Single endpoint: `POST /api/v1/generate-pr`
- Request validation via Pydantic models
- Async request handling with job queue return

2. Job Queue (`Celery + Redis`)

- Decouples long-running pipeline from HTTP response
- Returns `job_id` immediately; client polls `GET /api/v1/jobs/{job_id}`

3. Pipeline Orchestrator (`pipeline/orchestrator.py`)

- Coordinates all pipeline stages sequentially
- Owns error handling and cleanup lifecycle

4. Pipeline Stages (each isolated module)

- `RepoCloner` — git clone via `GitPython` into temp directory
- `CodeAnalyzer` — walks AST, extracts structure (files, classes, functions, dependencies)
- `ImplementationGenerator` — calls LLM (OpenAI GPT-4) with codebase context + feature desc
- `TestGenerator` — second LLM call scoped to generated implementation
- `PRSubmitter` — creates branch, commits, opens PR via `PyGitHub`
- `Cleanup` — always runs via `finally` block, removes temp dirs

Data Flow

```
Client → FastAPI → Celery Task → Orchestrator
  → RepoCloner (temp dir)
  → CodeAnalyzer (context object)
  → ImplementationGenerator (LLM) → code files
  → TestGenerator (LLM) → test files
  → PRSubmitter → PR URL
  → Cleanup (temp dir removed)
  → Job result stored in Redis
Client polls → returns PR URL
```

Key Tech Choices

Choice	Reason
FastAPI	Async, automatic OpenAPI docs
Celery+Redis	Pipeline takes 30-120s; async jobs essential
GPT-4 API	Best code generation quality
GitPython	Programmatic git without subprocess fragility
`tempfile.mkdtemp`	OS-managed isolation per job

Scaling Concerns

- **LLM token limits**: chunk large repos; send only relevant files via semantic similarity filter

- **Concurrent jobs**: scale Celery workers horizontally; Redis handles coordination
- **GitHub rate limits**: use per-user OAuth tokens, not a single service token
- **Temp disk space**: enforce max repo size (e.g., 500MB); reject oversized repos early

---

💻 Developer

AI-Powered PR Generation Pipeline — Full Implementation

Project Structure

```
pr_pipeline/
├── main.py                 # FastAPI app entrypoint
├── config.py               # Settings & env vars
├── models.py               # Pydantic request/response models
├── tasks.py                # Celery task definitions
├── pipeline/
│   ├── __init__.py
│   ├── orchestrator.py     # Coordinates all stages
│   ├── cloner.py           # Git clone stage
│   ├── analyzer.py         # Codebase structure analysis
│   ├── generator.py        # LLM code + test generation
│   └── submitter.py        # GitHub PR creation
├── utils/
│   ├── __init__.py
│   ├── cleanup.py          # Temp file management
│   ├── token_counter.py    # LLM context window management
│   └── logger.py           # Structured logging
├── requirements.txt
└── docker-compose.yml
```

---

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
celery==5.4.0
redis==5.0.4
gitpython==3.1.43
PyGithub==2.3.0
openai==1.30.1
pydantic==2.7.1
pydantic-settings==2.3.0
tiktoken==0.7.0
python-dotenv==1.0.1
httpx==0.27.0
tenacity==8.3.0
```

---

config.py

```python
from pydantic_settings import BaseSettings
from functools import lru_cache
from typing import Optional


class Settings(BaseSettings):
    # API
    api_host: str = "0.0.0.0"
    api_port: int = 8000
    api_secret_key: str = "change-me-in-production"

    # Redis / Celery
    redis_url: str = "redis://localhost:6379/0"
    celery_result_ttl: int = 86400  # 24 hours in seconds

    # OpenAI
    openai_api_key: str
    openai_model: str = "gpt-4o"
    openai_max_tokens: int = 4096
```

```python
    # Max tokens we'll send as context (leaves room for response)
    openai_context_limit: int = 100_000

    # GitHub
    github_token: str
    github_base_url: str = "https://api.github.com"

    # Pipeline constraints
    max_repo_size_mb: int = 500
    max_files_analyzed: int = 150
    temp_dir_base: str = "/tmp/pr_pipeline"

    # Relevant file extensions for analysis
    code_extensions: list[str] = [
        ".py", ".js", ".ts", ".jsx", ".tsx",
        ".java", ".go", ".rb", ".rs", ".cpp",
        ".c", ".h", ".cs", ".php", ".swift",
    ]

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"


@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

---

models.py

```python
from pydantic import BaseModel, HttpUrl, field_validator
from typing import Optional
from enum import Enum
import re


class JobStatus(str, Enum):
    PENDING = "pending"
    RUNNING = "running"
```

```
    SUCCESS = "success"
    FAILED = "failed"


class GeneratePRRequest(BaseModel):
    repo_url: str
    feature_description: str
    base_branch: str = "main"
    # Caller's GitHub token overrides the service default (multi-tenant support)
    github_token: Optional[str] = None

    @field_validator("repo_url")
    @classmethod
    def validate_github_url(cls, v: str) -> str:
        pattern = r"^https://github\.com/[\w.\-]+/[\w.\-]+(
```

🔒 Security Reviewer

Security Review: AI-Powered PR Generation Pipeline

Overall Security Posture

This service is a high-value attack target: it combines arbitrary code execution risk (cloning/analyzing untrusted repos), credential exposure (GitHub tokens, OpenAI keys), LLM prompt injection, and automated write access to production repositories — any single failure has severe blast radius.

Risk Registry

#	Risk	Severity	Component
1	Malicious repo triggers path traversal / symlink attack during clone+analysis	**Critical**	`cloner.py`, `analyzer.py`
2	Prompt injection via repo content hijacks LLM to generate malicious PR code	**Critical**	`generator.py`
3	GitHub tokens stored/logged in plaintext; leaked via job status API	**High**	`models.py`, `tasks.py`, `logger.py`
4	No authentication on REST API — anyone can trigger pipeline against any repo	**High**	`main.py`
5	LLM-generated code committed without security scanning (supply chain risk)	**High**	`submitter.py`

Risk 1 — Critical: Path Traversal & Symlink Attack via Malicious Repository

The Problem

A crafted repository can contain symlinks pointing to host filesystem paths (`/etc/passwd`, service credentials, other tenant temp dirs). The analyzer's directory walk will **follow them**, leaking file contents directly into the LLM context — which then potentially echoes them into the PR description or generated code.

```
# analyzer.py — VULNERABLE: blindly walks all paths
for root, dirs, files in os.walk(self.repo_path):
    for file in files:
        filepath = os.path.join(root, file)
        with open(filepath, "r") as f:  # Will follow symlinks!
            content = f.read()
```

Attack scenario:

```
# Attacker creates repo containing:
ln -s /etc/passwd credentials.txt
ln -s /tmp/pr_pipeline/ other_jobs/   # Access other tenants' clones
ln -s /root/.ssh/id_rsa deploy_key.txt
# Analyzer reads these → LLM receives host secrets → PR description leaks them
```

The Fix

```python
# utils/path_security.py
import os
import pathlib
from typing import Iterator
from utils.logger import get_logger

logger = get_logger(__name__)


def is_safe_path(base_dir: str, target_path: str) -> bool:
    """
    Verify target_path is strictly inside base_dir.
    Resolves all symlinks BEFORE the containment check.
    """
    try:
        base_resolved = pathlib.Path(base_dir).resolve(strict=True)
        target_resolved = pathlib.Path(target_path).resolve(strict=True)
        # is_relative_to is Python 3.9+; use str prefix check as fallback
        return str(target_resolved).startswith(str(base_resolved) + os.sep)
    except (OSError, ValueError):
        # resolve(strict=True) raises OSError if path doesn't exist
        # Treat non-existent resolved paths as unsafe
        return False


def safe_walk(base_dir: str, max_depth: int = 10) -> Iterator[tuple[str, str]]:
    """
    Walk directory yielding (relative_path, absolute_path) tuples.
    - Refuses to follow symlinks
    - Enforces max traversal depth
    - Validates every path is within base_dir
    """
    base_path = pathlib.Path(base_dir).resolve(strict=True)
```

```
    for root, dirs, files in os.walk(base_dir, followlinks=False):  # followlinks=F
        # Enforce depth limit
        root_path = pathlib.Path(root)
        depth = len(root_path.relative_to(base_path).parts)
        if depth > max_depth:
            dirs.clear()  # Prune traversal — modifying dirs in-place stops descent
            logger.warning("max_depth_exceeded", path=str(root_path), depth=depth)
```

🧪 QA Engineer

QA Engineering: Complete Test Suite for PR Generation Pipeline

Test Architecture Overview

```
tests/
├── conftest.py                    # Shared fixtures & mocks
├── unit/
│   ├── test_cloner.py             # RepoCloner unit tests
│   ├── test_analyzer.py           # CodeAnalyzer unit tests
│   ├── test_generator.py          # LLM generation unit tests
│   ├── test_submitter.py          # PRSubmitter unit tests
│   ├── test_orchestrator.py       # Pipeline orchestration unit tests
│   ├── test_models.py             # Pydantic model validation unit tests
│   ├── test_path_security.py      # Security utility unit tests
│   └── test_cleanup.py            # Cleanup utility unit tests
├── integration/
│   ├── test_api_endpoints.py      # REST API integration tests
│   ├── test_pipeline_flow.py      # End-to-end pipeline integration tests
│   └── test_celery_tasks.py       # Celery task integration tests
├── security/
│   └── test_security_controls.py  # Security-focused test cases
```

```
└── load/
    └── locustfile.py              # Load testing scenarios
```

---

tests/conftest.py

```python
import pytest
import tempfile
import os
import shutil
from pathlib import Path
from unittest.mock import MagicMock, AsyncMock, patch
from fastapi.testclient import TestClient
from celery import Celery

# ── App imports ──────────────────────────────────────────────────────────
from main import app
from config import Settings
from models import GeneratePRRequest, JobStatus
from pipeline.orchestrator import PipelineOrchestrator
from pipeline.cloner import RepoCloner
from pipeline.analyzer import CodeAnalyzer
from pipeline.generator import ImplementationGenerator
from pipeline.submitter import PRSubmitter


# ── Settings Override ────────────────────────────────────────────────────
@pytest.fixture(scope="session")
def test_settings() -> Settings:
    """Override production settings for test environment."""
    return Settings(
        openai_api_key="sk-test-key-not-real",
        github_token="ghp_test_token_not_real",
        redis_url="redis://localhost:6379/1",  # DB 1 = test isolation
        temp_dir_base="/tmp/pr_pipeline_test",
        max_repo_size_mb=10,                   # Tight limit for tests
        max_files_analyzed=20,
    )
```

```python
# ── Temp Directory Management ──────────────────────────────────────────────
@pytest.fixture
def temp_dir():
    """Create a fresh temp dir per test; guaranteed cleanup."""
    d = tempfile.mkdtemp(prefix="pr_pipeline_test_")
    yield d
    shutil.rmtree(d, ignore_errors=True)


@pytest.fixture
def fake_repo(temp_dir):
    """
    Construct a realistic fake Python repository on disk
    with proper structure, real Python files, and a git repo.
    """
    repo_dir = Path(temp_dir) / "fake_repo"
    repo_dir.mkdir()

    # --- Directory structure ---
    (repo_dir / "src").mkdir()
    (repo_dir / "src" / "api").mkdir()
    (repo_dir / "src" / "models").mkdir()
    (repo_dir / "tests").mkdir()
    (repo_dir / ".git").mkdir()            # Simulate initialized git repo

    # --- Python source files ---
    (repo_dir / "src" / "__init__.py").write_text("")
    (repo_dir / "src" / "api" / "__init__.py").write_text("")
    (repo_dir / "src" / "api" / "routes.py").write_text(
        """
from fastapi import APIRouter
router = APIRouter()

@router.get("/health")
def health_check():
    return {"status": "ok"}

@router.get("/users/{user_id}")
def get_user(user_
```

⭐ Tech Lead

Tech Lead Final Review: AI-Powered PR Generation Pipeline

Executive Summary

The team has produced solid foundational work. The architecture is sound, the security reviewer caught genuinely critical issues, and the QA engineer built a comprehensive test structure. However, there are **4 hard blockers** that prevent shipping as-is, **3 significant gaps** the developer left incomplete, and several areas where the pieces don't connect cleanly. Here is everything needed to close the gap.

What the Team Got Right — Keep It

Area	What's Good
Architecture	Celery+Redis async job pattern is exactly right for 30-120s pipelines
Security	`safe_walk`, symlink detection, path traversal fixes are production-grade
Models	Pydantic validation with GitHub URL regex is solid
Test structure	conftest fixtures, fake_repo builder, test isolation via Redis DB 1
Config	`lru_cache` settings, `.env` support, per-user token override
Cleanup	`finally` block lifecycle pattern is correct

Hard Blockers — Cannot Ship Without These

Blocker 1: Developer Left 3 Core Pipeline Files Unimplemented

The developer's submission was **cut off mid-code**. `models.py` ends mid-validator. The following files have **zero implementation** and are referenced everywhere:

- `pipeline/cloner.py`
- `pipeline/analyzer.py`
- `pipeline/generator.py`
- `pipeline/submitter.py`
- `pipeline/orchestrator.py`
- `tasks.py`
- `main.py`

Here is the complete implementation:

`main.py`

```
import uuid
from fastapi import FastAPI, HTTPException, Depends, Security, status
from fastapi.security.api_key import APIKeyHeader
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager

from config import get_settings, Settings
from models import (
    GeneratePRRequest,
    GeneratePRResponse,
    JobStatusResponse,
    JobStatus,
)
from tasks import run_pipeline_task
from utils.logger import get_logger

logger = get_logger(__name__)
API_KEY_HEADER = APIKeyHeader(name="X-API-Key", auto_error=True)
```

```python
def verify_api_key(
    api_key: str = Security(API_KEY_HEADER),
    settings: Settings = Depends(get_settings),
) -> str:
    """Constant-time API key comparison to prevent timing attacks."""
    import hmac
    if not hmac.compare_digest(api_key, settings.api_secret_key):
        logger.warning("invalid_api_key_attempt", provided_key_prefix=api_key[:6])
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Invalid API key",
        )
    return api_key


@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    import os
    os.makedirs(settings.temp_dir_base, exist_ok=True)
    logger.info("service_startup", temp_dir=settings.temp_dir_base)
    yield
    logger.info("service_shutdown")


app = FastAPI(
    title="PR Generation Pipeline",
    version="1.0.0",
    description="AI-powered pull request generation from feature descriptions",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],        # Lock this down per deployment environment
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)


@app.post(
    "/api/v1/generate-pr",
```

```
    response_model=GeneratePRResponse,
    status_code=status.HTTP_202_ACCEPTED,
    dependencies=[Depends(verify_api_key)],
)
async def generate_pr(request: GeneratePRRequest) -> GeneratePRResponse:
    """
    Accept a repo URL + feature description.
    Enqueue the pipeline job and return a job_id for polling.
    """
    job_id = str(uuid
```

✅ Team review complete!